

Code Buddy: Technical Documentation

1. Project Overview

Code Buddy is an autonomous AI agent designed to bridge the gap between Large Language Models (LLMs) and local development environments. Built using **LangChain** and **Gemini 1.5 Flash**, the agent acts as a "resident architect" that can read source code, analyze patterns, and suggest refactored improvements through structured tool-calling.

2. Tech Stack

Component	Technology	Purpose
Language	Python 3.12+	Core logic and scripting.
LLM	Google Gemini 2.5 Flash	Reasoning engine and code analysis.
Framework	LangChain	Orchestrating agentic workflows and tool binding.
Environment	Python-Dotenv	Secure management of API keys.
Architecture	AgentExecutor	Managing the Thought-Action-Observation loop.

3. Development Hurdles & Solutions

The "Ghost" Environment Conflict

- **Hurdle:** The agent was failing to import core libraries despite successful installations. Investigation revealed a "shadow" Python installation on the system path (Desktop) was overriding the local `.venv`.

- **Solution:** Implemented strict environment isolation. Reset the project using a clean `.venv` and verified paths using `where python` to ensure the agent runs within a contained ecosystem.

LangChain Namespace Shifting

- **Hurdle:** Rapid updates in the LangChain ecosystem caused `ImportErrors` for `AgentExecutor`.
 - **Solution:** Transitioned to explicit module imports and updated class names (e.g., using `ChatGoogleGenerativeAI` instead of the legacy `GoogleGenAI`).
-

4. Core Logic Workflow

The agent operates on a **ReAct** (Reason + Act) loop:

1. **Input:** User provides a code-related query or file path.
 2. **Thought:** The agent determines if it needs to read a file or use a tool.
 3. **Action:** The agent calls a specific tool (e.g., `list_sandbox_files`).
 4. **Observation:** The tool returns real-world data (file contents).
 5. **Final Response:** The agent synthesizes the data to provide a refactored solution.
-

5. Pseudocode

Python

```
FUNCTION create_code_agent():
    # Initialize the "Brain"
    llm = Load(ChatGoogleGenerativeAI, model="gemini-2.5-flash")

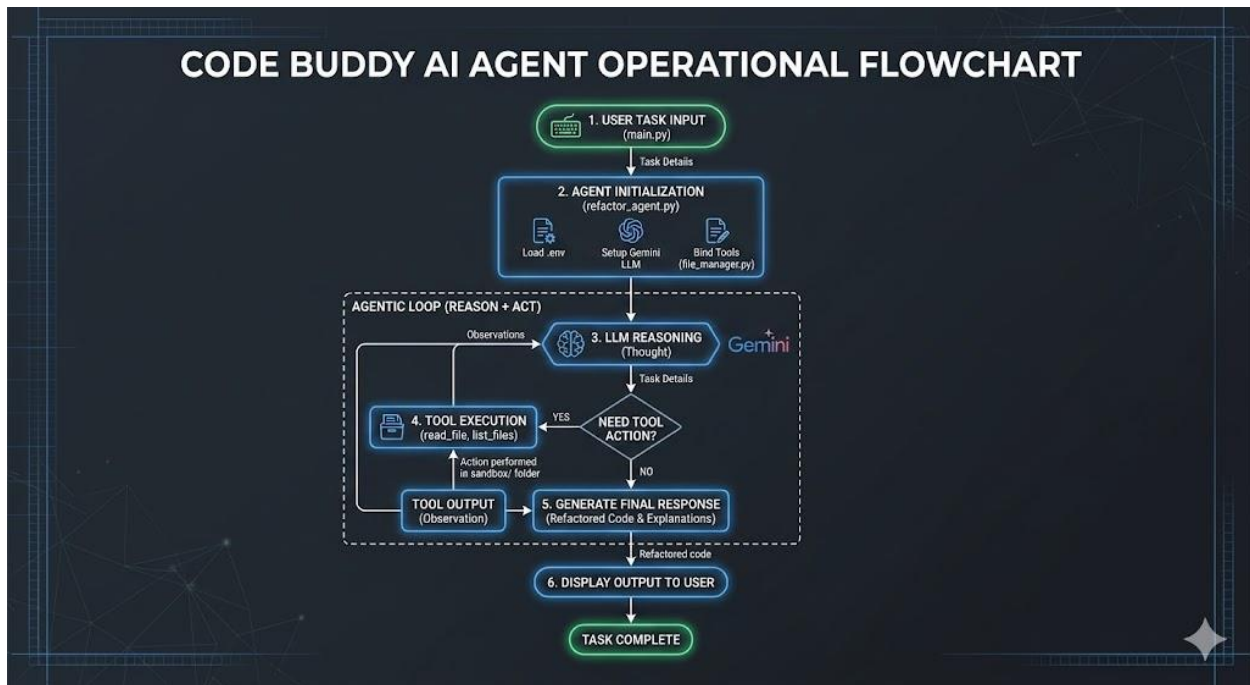
    # Define the "Hands" (Tools)
    tools = [read_file, list_files, analyze_syntax]

    # Create the Instructions
    prompt = Template("System: You are an AI Architect. Human: {input}")

    # Assemble the Agent
    agent = create_tool_calling_agent(llm, tools, prompt)

    RETURN AgentExecutor(agent, tools, verbose=True)

START:
    Initialize .env (API_KEYS)
    agent = create_code_agent()
    agent.run("Refactor the login logic in sandbox/auth.py")
```



6. System Modules

- **main.py:** The entry point. Handles the user interface and environment initialization.
- **agents/refactor_agent.py:** The "Intelligence" module. Contains the agent definition and prompt engineering.
- **tools/file_manager.py:** The "Utility" module. Provides the agent with safe access to the local filesystem.
- **sandbox/:** The "Working Directory." A restricted folder where the agent is allowed to read and analyze code.

7. Safety & Best Practices

- **Environment Security:** All sensitive credentials (API keys) are stored in `.env` and ignored by Git to prevent leakages.
- **Sandboxing:** Tools are restricted to a specific `sandbox/` directory. The agent cannot access system-level files or sensitive OS paths.
- **Type Hinting:** The project utilizes Python type hints (`mypy` compatible) to ensure backend stability and code maintainability.

Author:

Developed by: Muhammad Jami Ahad

